Guia de Consulta Técnica: Estruturas de Dados e Ordenação

Manual Completo de Algoritmos, Complexidades e Implementações Práticas

UNIDADE 1: ESTRUTURAS DE DADOS LINEARES

Conceito Base: Estruturas de dados lineares organizam elementos sequencialmente, onde cada elemento possui um antecessor e um sucessor direto (exceto o primeiro e o último).

1. Pilha (Stack - LIFO: Last In, First Out)

O último elemento inserido é o primeiro a ser removido. Operações principais: `push` (empilhar) e `pop` (desempilhar), ambas `O(1)`.

```
// Implementação de Pilha Genérica em Java
public class Pilha<T> {
    private static class No<T> {
        T dado;
        No<T> proximo;
        No(T dado) { this.dado = dado; }
    }

    private No<T> topo;

    public void push(T item) {
        No<T> novo = new No<>(item);
        novo.proximo = topo;
        topo = novo;
    }

    public T pop() {
        if (topo == null) throw new IllegalStateException("Pilha Vazia");
        T item = topo.dado;
        topo = topo.proximo;
        return item;
    }

    public boolean isEmpty() { return topo == null; }
}
```

2. Fila (Queue - FIFO: First In, First Out)

O primeiro elemento inserido é o primeiro a ser removido. Operações principais: `enqueue` (enfileirar) e `dequeue` (desenfileirar), ambas `O(1)`.

```
// Implementação de Fila Genérica em Java
public class Fila<T> {
    private static class No<T> {
        T dado;
        No<T> proximo;
        No(T dado) { this.dado = dado; }
    }

    private No<T> inicio, fim;

    public void enqueue(T item) {
        No<T> novo = new No<>(item);
        if (fim != null) fim.proximo = novo;
        fim = novo;
        if (inicio == null) inicio = fim;
    }

    public T dequeue() {
        if (inicio == null) throw new IllegalStateException("Fila Vazia");
        T item = inicio.dado;
        inicio = inicio.proximo;
        if (inicio == null) fim = null;
        return item;
    }
}
```

3. Lista Encadeada (Linked List)

Coleção linear de nós onde cada elemento aponta para o próximo. Permite inserção e remoção dinâmicas sem realocação de memória contígua.

```
// Implementação de Lista Simplesmente Encadeada em Java
public class ListaEncadeada<T> {
    private static class No<T> {
        T dado;
        No<T> proximo;
        No(T dado) { this.dado = dado; }
    }

    private No<T> cabeca;

    public void inserirInicio(T dado) {
        No<T> novo = new No<>(dado);
        novo.proximo = cabeca;
        cabeca = novo;
    }

    public void remover(T chave) {
        if (cabeca == null) return;
        if (cabeca.dado.equals(chave)) {
            cabeca = cabeca.proximo;
            return;
        }
        No<T> atual = cabeca;
        while (atual.proximo != null && !atual.proximo.dado.equals(chave)) {
            atual = atual.proximo;
        }
        if (atual.proximo != null) {
            atual.proximo = atual.proximo.proximo;
        }
    }
}
```

UNIDADE 2: MÉTODOS BÁSICOS DE ORDENAÇÃO (O(N²))

1. Ordenação por Seleção (Selection Sort)

Encontra repetidamente o menor elemento da sublista não ordenada e o move para o início. Invariante: a sublista à esquerda está sempre ordenada.

```
public class SelectionSort {
    public static void sort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) {
                    minIdx = j;
                }
            }
            int temp = arr[minIdx];
            arr[minIdx] = arr[i];
            arr[i] = temp;
        }
    }
}
```

2. Método da Bolha (Bubble Sort)

Compara iterativamente pares adjacentes e realiza a troca se estiverem fora de ordem. Os maiores elementos "flutuam" para o final a cada passagem.

```
public class BubbleSort {
    public static void sort(int[] arr) {
        int n = arr.length;
        boolean trocou;
        for (int i = 0; i < n - 1; i++) {
            trocou = false;
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    trocou = true;
                }
            }
            if (!trocou) break; // Otimização para vetores já ordenados
        }
    }
}
```

3. Ordenação por Inserção (Insertion Sort)

Constrói o vetor final ordenado um item por vez, inserindo cada novo elemento na posição correta da sublista já ordenada. Extremamente eficiente para pequenos conjuntos ou dados quase ordenados.

```java
public class InsertionSort {
    public static void sort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int chave = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > chave) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = chave;
        }
    }
}
```

UNIDADE 3: ALGORITMOS AVANÇADOS DE ORDENAÇÃO (O(N LOG N))

1. Merge Sort (Divisão e Conquista)

Divide o vetor recursivamente ao meio até subvetores unitários e combina-os de forma ordenada.

```java
public class MergeSort {
    public static void sort(int[] arr, int l, int r) {
        if (l < r) {
            int m = l + (r - l) / 2;
            sort(arr, l, m);
            sort(arr, m + 1, r);
            merge(arr, l, m, r);
        }
    }

    private static void merge(int[] arr, int l, int m, int r) {
        int n1 = m - l + 1, n2 = r - m;
        int[] L = new int[n1], R = new int[n2];
        System.arraycopy(arr, l, L, 0, n1);
        System.arraycopy(arr, m + 1, R, 0, n2);
        int i = 0, j = 0, k = l;
        while (i < n1 && j < n2)
            arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }
}
```

2. Quick Sort (Divisão por Pivô)

Escolhe um elemento como pivô e particiona o vetor de modo que elementos menores fiquem à esquerda e maiores à direita.

```java
public class QuickSort {
    public static void sort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = particionar(arr, low, high);
            sort(arr, low, pi - 1);
            sort(arr, pi + 1, high);
        }
    }

    private static int particionar(int[] arr, int low, int high) {
        int pivo = arr[high], i = (low - 1);
        for (int j = low; j < high; j++) {
            if (arr[j] < pivo) {
                i++;
                int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
            }
        }
        int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
        return i + 1;
    }
}
```

3. Heap Sort (Baseado em Árvore/Heap)

Converte o vetor em uma Heap Máxima e extrai o maior elemento sucessivamente.

```java
public class HeapSort {
    public static void sort(int[] arr) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
        for (int i = n - 1; i > 0; i--) {
            int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;
            heapify(arr, i, 0);
        }
    }

    private static void heapify(int[] arr, int n, int i) {
        int maior = i, l = 2 * i + 1, r = 2 * i + 2;
        if (l < n && arr[l] > arr[maior]) maior = l;
        if (r < n && arr[r] > arr[maior]) maior = r;
        if (maior != i) {
            int swap = arr[i]; arr[i] = arr[maior]; arr[maior] = swap;
            heapify(arr, n, maior);
        }
    }
}
```

MATRIZ COMPARATIVA DE COMPLEXIDADE

Algoritmo	Tempo (Melhor)	Tempo (Médio)	Tempo (Pior)	Espaço	Estável?
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$	Não
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Sim
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Sim
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$	Sim
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$	Não
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$	Não